

Sortieren - allgemeine Bemerkungen

Mehr als 25% der kommerziell verbrauchten Rechenzeit entfällt auf Sortiervorgänge. Aus diesem Grund werden noch immer Anstrengungen unternommen, möglichst effiziente Verfahren zum Sortieren von Daten mit Hilfe von Computern zu entwickeln.



Wikimedia Commons

Sortierproblem

Eine Menge von Datensätzen ist gegeben.

Jeder Datensatz besitzt einen Schlüssel.

Zwischen den Schlüsseln ist eine Ordnungsrelation $\leq$ erklärt.

Außer der Schlüsselkomponente können Sätze weitere Komponenten als eigentliche Informationen enthalten.

Schlüssel sind ganzzahlig, z. B. Hausnummer, Artikelnummer, Rechnungsnummer,....



Sortierproblem

Gegeben ist eine Folge von Datensätzen $a_1, \dots, a_n$.

Jeder Datensatz a_i hat einen Schlüssel $a_i.\text{key}$.

Gesucht ist eine Permutation p der Zahlen von 1 bis n derart, dass die Umordnung der Sätze gemäß p die Schlüssel in aufsteigender Reihenfolge wiedergibt:

$$a_1.\text{key} \leq a_2.\text{key} \leq \dots \leq a_n.\text{key}.$$

Sortierproblem

Sortieren besteht aus 2 Teilproblemen:

1. Informationsbeschaffungsproblem

Vergleich zwischen je zwei Schlüsseln

- $k \leq k'$ oder $k > k'$

2. Datentransportproblem

- Vertauschen zweier benachbarter oder zweier beliebiger Sätze
- das Verschieben einzelner Sätze um eine feste oder beliebige Distanz
- das Platzieren von Datensätzen an eine bestimmte Speicheradresse

Sortieren der Tabelle

Unterscheide:

1. Internes Sortieren: Es wird im Hauptspeicher sortiert.
2. Externes Sortieren: Es wird auf externen Massenspeichern (Festplatte) sortiert.



Sortiervverfahren

- Sortieren durch Einfügen
- Sortieren durch Auswählen
- Sortieren durch Austauschen

benötigen keinen nennenswerten zusätzlichen Speicher

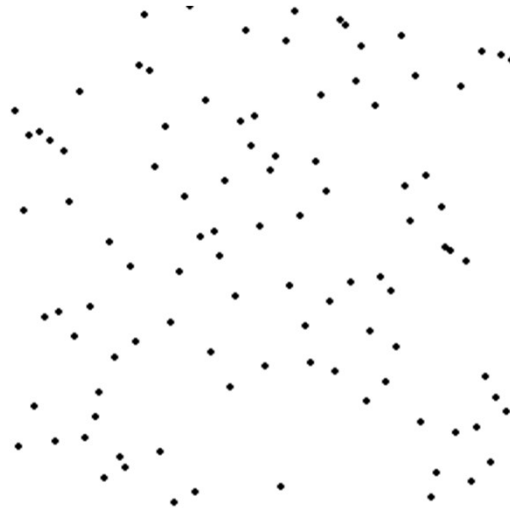
Für die Komplexitätsanalyse wichtige Kriterien:

Anzahl Schlüsselvergleiche $C(n)$

Anzahl der Recordbewegungen $M(n)$



Insertionsort



Selectionsort



Exchangesort

Sortiervverfahren

- elementare Sortiervverfahren (direktes Sortieren)
 - $O(n^2)$
 - Exchangesort (=Bubblesort), Insertionsort, Selectionsort
- schnelle Sortiervverfahren
 - $O(n \log n)$
 - Quicksort, Mergesort, Heapsort

Insertionsort - Sortieren durch Einfügen

```
template <class Tr>
void Table <Tr>::InsertionSort()
{
    int i,j;
    for (i=1; i<n; i++)
    {
        an=a[i];
        j=i;
        while (j>0 && an.key<a[j-1].key)
        {
            a[j]=a[j-1];
            j--;
        }
        a[j]=an;
    }
}
```


Komplexität von Insertionsort

- Anzahl der Vergleiche:
 - für jeden Wert $i=1, \dots, n-1$:
 - minimal 1 Vergleich $\rightarrow n-1$ Vergleiche
 - maximal i Vergleiche
 - $C_{\text{best}} \in O(n)$
 - $C_{\text{worst}} = 1+2+3+\dots+n-1 = n*(n-1)/2 \in O(n^2)$ maximale Vergleiche
- Anzahl Recordbewegungen:
 - für jeden Wert $i=1, \dots, n-1$:
 - in der äußeren Schleife 2 Zuweisungen
 - in der inneren Schleife
 - minimal 0 Zuweisungen
 - maximal i Zuweisungen
 - $M_{\text{best}} = 2*(n-1) \in O(n)$
 - $M_{\text{worst}} = 2*(n-1) + n*(n-1)/2 = n^2/2 + 3/2n - 2 \in O(n^2)$

Selectionsort - Sortieren durch Auswählen

```
template <class Tr>
void Table <Tr>::SelectionSort()
{
    int i,j,imin;
    for (i=0;i<n-1;i++)
    {
        imin=i;
        for (j=i+1;j<n;j++)
            if (a[j].key<a[imin].key) imin=j;
        swap(a[i],a[imin]);          //vertauschen
    }
}
```

Komplexität von Selectionsort

- Anzahl der Vergleiche:
 - für jeden Wert $i=0, \dots, n-2$:
 - genau $n-1-i$ Vergleiche
 - $C_t = 1+2+3+\dots+n-1 = n*(n-1)/2 \in O(n^2)$
- Anzahl Recordbewegungen:
 - für jeden Wert $i=0, \dots, n-2$:
 - genau 3 Zuweisungen (Vertauschung)
 - $M = 3*(n-1) \in O(n)$

Folgerungen

- Selectionsort und Insertionsort sind i.a. von derselben Komplexitätsklasse
- Für teilweise vorsortierte arrays kann Insertionsort schneller sein, da im besten Fall $O(n)$
- Im allgemeinen ist Selectionsort schneller, da nur $O(n)$ Zuweisungen nötig sind.

Exchangesort = Bubblesort

Sortieren durch Austauschen

```
template <class Tr>
void Table <Tr>::ExchangeSort()
{
    int i,j;
    for (i=0;i<n-1;i++)
    {
        for (j=n-2;j>=i;j--)
            if (a[j].key>a[j+1].key)
            {
                swap(a[j],a[j+1]);
            }
    }
}
```

Komplexität von Exchangesort

- Anzahl der Vergleiche:
 - für jeden Wert $i=0, \dots, n-2$:
 - genau $n-1-i$ Vergleiche
 - $C_t = 1+2+3+\dots+n-1 = n*(n-1)/2 \in O(n^2)$
- Anzahl Recordbewegungen:
 - bestenfalls (falls alles sortiert) $=0$
 - schlechtesten Falls für jeden Wert $i=0, \dots, n-2, j=i..n-2$:
 - genau 3 Zuweisungen (Vertauschung)
 - $M = 3*n*(n-1)/2 \in O(n^2)$

QuickSort

- Nach C.A.R. Hoare (1961)
- Anwendung des Divide and Conquer-Prinzips
 - Sortieren von Tabellen wird auf Sortieren von Teiltabellen zurückgeführt
- Beschreibung des Algorithmus:

Wiederhole folgende Schritte bis die Teiltabelle aus einem Element bestehen:

1. Auswahl des Pivot-Elements
2. Zerlegung der Tabelle in 2 Teiltabellen T_1 und T_2 so daß:
 - Für alle Elemente aus T_1 $\text{Element.key} \leq \text{Pivot.key}$
 - Für alle Elemente aus T_2 $\text{Element.key} > \text{Pivot.key}$
 - Ergebnis: Pivot-Element steht an der richtigen Position
3. Rekursives Aufrufen der Quicksort-Funktion für die beiden Teiltabellen T_1 und T_2

QuickSort

Schritt 1:

- Auswahl des Pivot-Elements
- wird durch Methode Mean realisiert
- Auswahlstrategie leicht austauschbar

```
template <class Tr>
void Table<Tr>::Mean(int l,int r)
{
    int m=(l+r)/2; // zunächst einfach(fast) mittleres Element
    Swap(a[m],a[r]); // setze Pivotelement an das Tabellenende
}
```


QuickSort

```
template <class Tr>
int Table<Tr>::Partition(int l,int r)
{
    int i=l-1,j=r;
    Mean(l,r);
    while (i<j)
    {
        while(a[++i]<=a[r])
            if (i==r) break;
        while(a[--j]>a[r])
            if (j==l) break;
        if (i<j) Swap(a[i],a[j]);
    };
    Swap(a[r],a[i]);
    return i;
}
```

QuickSort

Rekursive Funktion Quicksort:

```
template <class Tr>
void Table<Tr>::QuicksortTable(int l,int r)
{
    int m;
    m=Partition(l,r);
    if (l<m-1) QuicksortTable(l,m-1);
    if (m+1<r) QuicksortTable(m+1,r);
}
```

Komplexität von Quicksort

hängt von der Wahl des Pivot-Elements ab:

- bestmögliche Laufzeit, falls jede Zerlegung auf gleich große Teilfolgen T_1 und T_2 führt:
 - im 1. Durchlauf wird 1 Element
 - im 2. Durchlauf werden 2 Elemente
 - im 3. Durchlauf werden 4 Elemente
 -
- Sortiert
- > Gesamtaufwand ist $n \log(n)$

Komplexität von Quicksort

hängt von der Wahl des Pivot-Elements ab:

- Die Laufzeit hängt davon ab, wie gut die Zerlegungsmethode “halbiert”.
 - Bester Fall:
 - Das Feld wird immer halbiert:
 - Schlechtester Fall:
 - Es wird immer nur 1 Element abgespalten.
 - schlechteste Laufzeit, falls Pivot immer das kleinste oder größte Element ist:
 - in jedem Durchlauf wird ein Element sortiert
- > Gesamtaufwand ist n^2

Komplexität von Quicksort

hängt von der Wahl des Pivot-Elements ab:

- im Allgemeinen $O(n \log n)$ Vergleiche und $O(n \log n)$ Recordbewegungen
- Mittlerer (erwarteter) Fall:
- $\Theta(n \log n)$ = (Analyse aufwendiger; vgl. Literatur)
- Plausibilitätsbetrachtung: QuickSort produziert eine Mischung aus “guten” und “schlechten” Zerlegungen.

Vorteile von Quicksort:

- leicht zu implementieren
- im schlechtesten Fall $O(n^2)$
- mit Median besser als n^2
- im allgemeinen $O(n \log n)$

Indirektes Sortieren

- Es werden nicht die records selbst, sondern die indices sortiert
- Indices werden z.B. im array `int s[i]` abgelegt.
 - `a[i]` unsortiertes array -> `a[s[i]]` sortiertes array

MergeSort

Sortieren durch Mischen

- Verschmelzung von 2 geordneten Teilfolgen zu einer Gesamtfolge (Zweiwege-Mischen)
- Anwendung auf Sortieralgorithmus
 - zunächst Mischen von Teilfolgen der Länge 1
 - jeweils 2 aufeinanderfolgende Zahlen werden in eine geordnete 2er-Folge abgelegt.
 - Dann Mischen von Teilfolgen der Länge 2
 - jeweils 2 aufeinanderfolgende 2er-Folgen werden in eine geordnete 4er-Folge abgelegt.
 -
- Für die Mischoperation wird ein zweites Datenfeld benötigt.

MergeSort

```
template <class Tr>
void Table<Tr>::Merge(int l, int m, int r)
{
    int i=1,j=m+1,k=0;
    Tr *b=new Tr[r-l+1];
    while (i<=m && j<=r)
    {
        if (a[i]<=a[j]) b[k]=a[i++];
        else b[k]=a[j++];
        k++;
    }
    if (i>m) for (;j<=r;j++) b[k++]=a[j];
    else for (;i<=m;i++) b[k++]=a[i];
    k=0;
    for (i=1;i<=r;i++) a[i]=b[k++];
    delete b[];
}
```


MergeSort

Rekursive Funktion Mergesort

```
template <class Tr>
void Table<Tr>::Mergesort(int l, int r)
{
    int m;
    if (r>l)
    {
        m=(l+r) /2;
        Mergesort(l,m);
        Mergesort(m+1,r);
        Merge(l,m,r);
    }
}
```

Komplexität von Mergesort

- Mergesort Algorithmus des Typs Divide and Conquer
- Falls die Verschmelzung von n Elementen die Zeit $O(n)$ und Mergesort für die Sortierung von n Elementen komplett $T(n)$ braucht ergibt sich folgende Laufzeit:

$m = (l+r)/2;$	// $O(1)$	elementare Operation
Mergesort(l, m);	// $T(n/2)$	linke Hälfte sortieren
Mergesort($m+1, r$);	// $T(n/2)$	rechte Hälfte sortieren
Merge(l, m, r);	// $O(n)$	sortierte Hälften verschmelzen

Komplexität von Mergesort

In jeder Schichte des Baums sind 2^i Felder der Größe $n/2^i$ zu mischen
dabei ist i mit $1 \leq i \leq \log_2(n)$ die Schichtnummer

Der Aufwand pro Schicht beträgt cn .

c = Konstante

Anzahl der Schichten in dem Baum:

$\log_2(n)$ Schichten.

Gesamtaufwand demnach: $cn \log_2(n)$.

Komplexität von Mergesort

Formal: Ist $T(n)$ die Laufzeit bei Problemgröße n , so gilt:

$$T(n) \leq \begin{cases} cn; & \text{falls } n = 2 \\ cn + 2T\left(\frac{n}{2}\right); & \text{falls } n > 2 \end{cases}$$

Damit ergibt sich durch vollständige Induktion über n :

$$T(n) \leq cn \log_2 n$$

Einfluß von Vorsortierung auf die Laufzeit

Laufzeitvergleiche der elementaren Sortialgorithmen

bei zufällig sortierten Daten:

	n=10000	n=20000	n=30000	n=40000
Exchange	1,7	6,7	14,7	26,1
Selection	0,8	3,1	7,1	12,6
Insertion	0,4	1,25	2,9	6,7

bei vorsortierten Daten:

	n=10000	n=20000	n=30000	n=40000
Exchange	0	0	0	0
Selection	0,75	3,1	7,1	12,6
Insertion	0	0	0	0

ExchangeSort und InsertionSort sind bei vorsortierten Daten effizienter.

Einfluß von Vorsortierung auf die Laufzeit

In der Praxis sind vorsortierter Daten sehr häufig, z.B.:

- unsortierter Datenbestand wird einmalig in ein Programm eingebracht und muss zuerst aufwändig sortiert werden.
- Es ändern sich danach nur wenige Datensätze:
 - Einige Datensätze werden neu eingebracht, einige werden geändert oder gelöscht
- die notwendige Neusortierung wird auf sehr gut vorsortierten Daten durchgeführt

In diesem häufigen Fall sind ExchangeSort und InsertionSort gut geeignete Algorithmen.

Verbesserte Varianten der einfachen Sortieralgorithmen

Shell Sort: Variante von *Insertion Sort*

Comb Sort: (Variante von *Bubble Sort*)

- **Sortieralgorithmen mit Vorsortierung:**
 - **Vorsortieren in Sequenzen:**
 - Zerlegung der gesamten Datenmenge in Teilsequenzen z.B. durch modulo-Operation des key-Feldes.
 - Sortiere diese Sequenzen mit einfachem Sortiervorgang vor
 - Aufwand beim abschließenden Haupt-Sortiervorgang ist wesentlich geringer.

Shell Sort

Shell Sort (nach Donald Shell) ist eine Variante von Insertion Sort

- Insertion Sort-Algorithmus wird durch eine natürliche Zahl h parametrisiert, die die Anzahl der vorsortierten Sequenzen bestimmt.
- Schrittweite h bestimmt die vorsortierten Teilsequenzen durch Modulo-Operation im Indexbereich des arrays
- Mehrere Sortiervorgänge werden durchgeführt für unterschiedliche Teilsequenzen
- Zum Schluss wird das komplette Feld mit Insertionsort sortiert ($h=1$)

InsertionSort auf Teilfolge modulo h

```
void insertionsort_h (int h)
{
    for (int is = 0; is<h; is++) // Startwerte für Teilfolgen
    {
        for (i=is+h,i<n,i+=h) // Insertionssort mit Schrittweite h
        {
            int an = a[i];    // kopiere einzusortierendes Element
            j=i;
            for (j = i ; (j > is && (a[j-h] > an); j-= h)
                a[j] = a[j-h]; // Suche Einfügepos. Modulo h
            a[j] = an; //Einfügen des einzusortierenden Elements
        }
    }
}
```

Shell Sort

Wahl der Schrittweite für Shellsort:

Donald Shell empfiehlt folgende rekursiv definierte Zahlenfolge:

$$h = 1, h_{i+1} = 3 * h_i + 1$$

Dies ergibt folgende Schrittweitenwahl:

$$h = \dots 1093, 364, 121, 40, 13, 4, 1$$

Am Schluss muss immer mit Schrittweite 1, also Insertionsort komplett sortiert werden

Beispiel: Feld der Länge 100:

1. Sortieren mit Schrittweite 40 (40 Sortiervorgänge)
2. Sortieren mit Schrittweite 13 (13 Sortiervorgänge)
3. Sortieren mit Schrittweite 4 (4 Sortiervorgänge)
4. Sortieren mit Schrittweite 1 (1 Sortiervorgänge)

Gesamt: 58 Sortiervorgänge -> schneller als einmal Sortieren?

Shell Sort

Korrektheit und Komplexitätsbetrachtungen

- Korrektheit von Shell Sort:
 - da der letzte Durchlauf InsertionSort mit $h = 1$ durchgeführt wird, ist Shell Sort korrekt
- Laufzeitverhalten von Shell Sort
 - besser als Insertion Sort $= O(n^2)$
 - $O(n^{3/2})$
 - Experimentell bestimmt $O(n^{5/4})$

Shell Sort

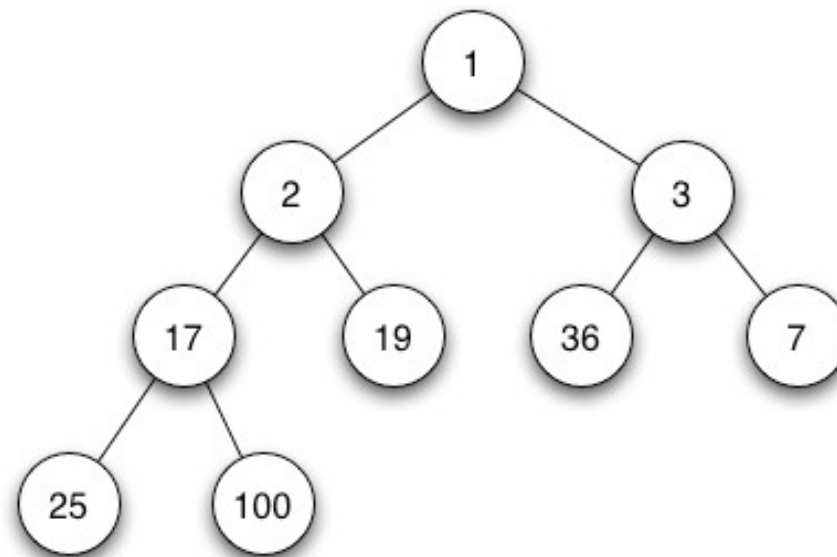
- Shell Sort nutzt aus, daß Insertion Sort nur geringen Aufwand verursacht, falls die Daten bereits (vor-) sortiert sind.
- Shell Sort gehört zu den populärsten Sortieralgorithmen.
- Vorteile:
 - schnell bei vorsortierten Daten
 - einfach zu programmieren

Andere Algorithmen mit Vorsortieren

- Die Vorsortier-Variante von Bubble Sort wird Comb Sort genannt.
- Auch bei Comb Sort wird in Sequenzen vorsortiert.
- Zahlenfolge für die Sequenzen ist dichter gelegt, z.B.
 $h_n = n/SF$, $h_{n-1} = h_n/SF$, $h_{n-2} = h_{n-1}/SF$, ..., h_0 ,
SF="Schrumpf-Faktor" = 1.3
- Beispiel für daraus resultierende Zahlenfolge: 11, 8, 6, 4, 3, 2, 1

Weitere Sortiervverfahren

- HeapSort
 - benutzt Datenstruktur Binärbaum
 - Schnelles Sortiervverfahren
 - Baumstruktur kann direkt auf gegebenem Array implementiert werden
 - > müssen zuerst Bäume einführen



Datenstruktur Baum

Definition:

Eine Menge von Knoten eines gegebenen Typs heißt Baum, wenn gilt:

- Es gibt einen ausgewählten Knoten (die Wurzel), der keinen Vorgänger hat
- Jeder andere Knoten hat genau einen Vorgängerknoten
- zu einem beliebigen Knoten führt genau eine von der Wurzel ausgehende Folge von Knoten (Weg)

Datenstruktur Baum

Definition (rekursiv):

- eine leere Menge von Knoten ist ein Baum
- ein einzelner Knoten ist ein Baum
- Iterationsvorschrift:
 - Gegeben Knoten r und k Bäume mit den Wurzeln $r_1, r_2, r_3, \dots, r_k$
 - Baum wird konstruiert, in dem r als Wurzel und $r_1, r_2, r_3, \dots, r_k$ als Nachfolger von r gewählt werden.

Datenstruktur Baum

Terminologie:

- Teilbaum
 - Teilbäume ergeben sich aus der rekursiven Definition
($r_1, r_2, r_3, \dots, r_k$ sind Teilbäume)
- die Anzahl der Nachfolger eines Knoten heißt Grad des Knotens
- Ein Knoten vom Grad 0 heißt Blatt des Baums
- Der höchste im Baum vorkommende Grad eines Knotens heißt Grad oder Ordnung des Baums.
- Ein Baum der Ordnung k heißt auch k -ärer Baum (Bin, Tri, Quartärbaum)

Datenstruktur Baum

Terminologie:

- Weg oder Pfad (path):
 - Weg ist eine Folge von Knoten $r_1, r_2, r_3, \dots, r_k$, wobei jeweils r_i der Vorgänger von r_{i+1} ist.
 - Die Weglänge ist die Anzahl der Knoten im Weg -1.
 - Zu jedem Knoten r gibt es genau einen Weg von der Wurzel zu diesem Knoten
 - Die Wege zu den Blattknoten nennt man Zweige
 - Alle Knoten, deren Weglänge (von der Wurzel ausgehend) gleich ist, liegen auf derselben Stufe
 - die größte Stufe im Baum wird als die Höhe des Baums bezeichnet

Datenstruktur Baum

Terminologie:

- Vorgänger/Nachfolger:
 - der Vorgänger eines Knotens wird als Vater bezeichnet
 - der Nachfolger eines Knotens wird als Kind bezeichnet
 - Kinder desselben Vaters heißen Geschwister
- Operationen auf Bäumen
 - Aufsuchen aller Knoten im Baum in einer bestimmten Reihenfolge (heißt Traversieren des Baums):
 - Zugriff auf einen oder alle Kinder eines gegebenen Knotens
 - Zugriff auf Vater eines Knotens
 - Einfügen, Löschen und Ersetzen von Knoten
- Wald:
 - eine Menge von Bäumen vom gleichen Grundtyp
- lineare Liste
 - entarteter Baum: jeder Knoten hat genau einen Teilbaum

Binärbäume

- Vorteile Binärbaum:
 - einfach abzuspeichern
 - viele Aufgaben lassen sich direkt auf Binärbäume zurückführen (Binärsuche!)
- Terminologie
 - vollständige binäre Bäume:
 - alle Knotenpositionen sind besetzt außer in der höchsten Stufe. Hier können Knoten fehlen
 - Wachstumsverhalten eines vollständigen Binärbaums: Verdoppelung der Knoten von Stufe zu Stufe

Binärbäume

- Höhe eines Binärbaums mit n Knoten (=Suchaufwand):
 - minimale Höhe $h = \log_2 n$
 - maximale Höhe $h = n - 1$
- vollständig ausgeglichener Baum
 - für jeden Knoten gilt:
 - die Differenz der Anzahl der Knoten im linken und rechten Teilbaum eines jeden Knotens ist höchstens 1

Binärbäume

- Darstellung eines Baums der Höhe h :
 - Darstellung auf natürliche Weise mittels Pointervariablen
 - Arraydarstellung in einem array a der Länge $2^{h+1}-1$ (2^{h+1})
 - sehr effizient für vollständige Binärbäume
 - Kanten müssen nicht gespeichert werden
 - Linkes Kind des Knotens $a[i]$ ist $a[2*i]$
 - Rechtes Kind des Knotens $a[i]$ ist $a[2*i+1]$
 - Vater des Knotens $a[i]$ ist $a[i/2]$
 - Wurzel ist $a[1]$
 - nicht besetzte Knoten müssen markiert werden (z.B. $a[i]=-1$)

Beispiele Binärbaum

- Stammbaum
- Begegnungen in einem Tennis- oder Fußballturnier (k.o.-System)
- arithmetischer Ausdruck

Operationen auf Binärbäumen

- Knoten besuchen (traversieren)
 - Preorder (WLR) (Wurzel, links, rechts)
 - Inorder (links, Wurzel, rechts)
 - Postorder (links, rechts, Wurzel)
- Beispiel arithmetischer Ausdruck:
 - Preorder ->Prefix
 - Inorder -> Infix
 - Postorder ->Postfix

ADT Binary Tree - Knoten

```
template <class Tr>
class TreeNode
{
public:
    TreeNode(Tr r)    : Trvalue(r),left(NULL),right(NULL) {}
    Tr Trvalue;        // Wert vom Typ Tr
    TreeNode *left,    // Zeiger auf linken Teilbaum
    *right;            // Zeiger auf rechten Teilbaum
    int path;          //Weglänge
    void SetSuccessors(TreeNode<Tr> *l,TreeNode<Tr> *r);
};
```

ADT Binary Tree - Positionszeiger

```
template <class Tr>
class TPosition
{
public:
    TPosition() { p=NULL; }
    bool TPos() { return (p) ? true : false; }
    bool TPos(TPosition pos)
        { return (pos.p==p) ? true : false; }
    TreeNode<Tr> *p;
};
```

ADT Binary Tree

Öffentlich sichtbare Funktionen:

```
template <class Tr>
class BinaryTree
{
public:
    BinaryTree();      //Konstruktor
    ~BinaryTree();     //Destruktor
    TPosition<Tr> Root(); // Wurzel des Baumes
    TPosition<Tr> Left(Tposition<Tr> pos); //linker Teilbaum von pos
    TPosition<Tr> Right(Tposition<Tr> pos); //rechter Tb von pos
```

ADT Binary Tree

Öffentlich sichtbare Funktionen:

```
void InsertRoot (Tr r); // fügt record r als Wurzel ein
void InsertLeft(Tr r, Tposition<Tr> pos); //fügt r als linken
                                     // Teilbaum an Knoten pos an.
void InsertRight(Tr r, Tposition<Tr> pos); //fügt r als rechten
                                     // Teilbaum an Knoten pos an.
Void DeleteTree(); //löscht den Baum
Tposition<Tr> DeleteNode(Tposition<Tr> pos); //entfernt Knoten pos
Tr GetNode(Tposition<Tr> pos) //liefert Record an der Position pos
void SetNode (RE &r, Tposition<Tr> pos); //ersetzt Record an
                                     //Position pos durch r
```

ADT Binary Tree

private Funktionen:

private:

```
TreeNode<Tr> *t; // Zeiger auf Wurzel
```

```
void DelTree(TPosition<Tr> pos); // Hilfsmethode zum  
Loeschen
```

ADT Binary Tree

Konstruktor: generiert einen leeren Binärbaum

```
template <class Tr>
BinaryTree<Tr>::BinaryTree() : t(NULL) { }

// Destruktor gibt Baum frei
template <class Tr>
BinaryTree<Tr>::~~BinaryTree()
{
    if (t) DeleteTree();
}
```

ADT Binary Tree

liefert Position der Wurzel des Baums

```
template <class Tr>
TPosition<Tr> BinaryTree<Tr>::Root( )
{
    TPosition<Tr> pos;
    pos.p=t;
    return pos;
}
```

ADT Binary Tree

liefert für Knoten pos die Position des linken Teilbaums

```
template <class Tr>
TPosition<Tr> BinaryTree<Tr>::Left(TPosition<Tr> pos)
{
    TPosition<Tr> pos1;
    if (pos.p) pos1.p=pos.p->left;
    return pos1;
}

// rechts: analog
```


ADT Binary Tree

fügt Record in r als Wurzel des Baums ein

```
template <class Tr>
void BinaryTree<Tr>::InsertRoot(Tr r)
{
    if (!t) t=new TreeNode<Tr>(r);    // Knoten mit Record r
}
```

ADT Binary Tree

fügt Record r als linken Teilbaum in Knoten pos ein

```
template <class Tr>
void BinaryTree<Tr>::InsertLeft(Tr r, TPosition<Tr> pos)
{
    if (t==NULL) return;
    TreeNode<Tr> *q=new TreeNode<Tr>(r); // Knoten mit Record r
    q->right=pos.p->left;
    pos.p->left=q;
}
// rechts analog
```

ADT Binary Tree

löscht alle Knoten im Baum

```
template <class Tr>
void BinaryTree<Tr>::DeleteTree()
{
    DelTree(Root());
    t=NULL;
}

template <class Tr>
void BinaryTree<Tr>::DelTree(TPosition<Tr> pos)
{
    if (pos.TPos())
    {
        DelTree(Left(pos));
        DelTree(Right(pos));
        delete pos.p;
    }
}
```

ADT Binary Tree

liefert Record an der Position pos

```
template <class Tr>
Tr BinaryTree<Tr>::GetNode(TPosition<Tr> pos)
{
    Tr r;
    if (pos.p) return pos.p->Trvalue;
    else return r;
}
```

ersetzt Record an der Position pos durch Record r

```
template <class Tr>
void BinaryTree<Tr>::SetNode(Tr &r, TPosition<Tr> pos)
{
    if (pos.p) pos.p->Trvalue=r;
}
```

ADT Binary Tree

weitere öffentlich sichtbare Funktionen:

```
void PreOrder(TPosition<Tr> pos);    //Traversieren in Preorder
void InOrder(TPosition<Tr> pos);      // Traversieren in Inorder
void PostOrder(TPosition<Tr> pos);    // Traversieren in Postorder
unsigned NumNodes(TPosition<Tr> pos); //Anzahl der Knoten im Baum
void Visit(Tr r);                     // Ausgabe des Knotens
```

ADT Binary Tree

Traversieren in Preorder

```
template <class Tr>
void BinaryTree<Tr>::PreOrder(TPosition<Tr> pos)
{
    if (pos.TPos())
    {
        Visit(GetNode(pos));
        PreOrder(Left(pos));
        PreOrder(Right(pos));
    }
}
```

ADT Binary Tree

Traversieren in Inorder

```
template <class Tr>
void BinaryTree<Tr>::InOrder(TPosition<Tr> pos)
{
    if (pos.TPos())
    {
        InOrder(Left(pos));
        Visit(GetNode(pos));
        InOrder(Right(pos));
    }
}
```

ADT Binary Tree

Traversieren in Postorder

```
template <class Tr>
void BinaryTree<Tr>::PostOrder(TPosition<Tr> pos)
{
    if (pos.TPos())
    {
        PostOrder(Left(pos));
        PostOrder(Right(pos));
        Visit(GetNode(pos));
    }
}
```


ADT Binary Tree

Anzahl der Knoten im Baum

```
template <class Tr>
unsigned BinaryTree<Tr>::NumNodes(TPosition<Tr> pos)
{
    if (pos.TPos())
        return NumNodes(Left(pos)) + NumNodes(Right(pos)) + 1;
    else return 0;
}
```

ADT Binary Tree

Ausgabe des Knotens

```
template <class Tr>
void BinaryTree<Tr>::Visit(Tr r)
{
    cout << r.GetKey() << " ";
}
```

Heap

Definition Heap:

Ein Heap ist ein vollständiger Binärbaum mit Heapeigenschaft:

Definition Heapeigenschaft:

Für jeden Knoten gilt: Vater ist jeweils kleiner als seine beiden Kinder.

Folgerung:

Für jeden Teilbaum eines Heaps gilt:

- in der Wurzel steht der minimale Schlüssel des Teilbaums
- die Folge der Schlüssel längs eines Weges von der Wurzel zum Blatt ist nicht fallend

Heapsort

Heapsort-Algorithmus:

- Herstellen der Heapeigenschaft
- An der Baumwurzel befindet sich das kleinste Element (Heapeigenschaft)
- Kleinstes Element wird entnommen und abgelegt
- Ersetzen der Wurzel durch das letzte Element in der untersten Reihe des Heaps
- ‚Absenken‘ der Wurzel:
 - Vertauschen mit dem kleineren Kind, bis Element auf eine passende Position ‚abgesunken‘ ist
 - Damit ist die Heap-Eigenschaft wieder erfüllt
- In der Ausgangssituation kann jeder Baum durch schrittweises Absenken der Knoten in einen Heap transferiert werden

Anwendungen des ADT Baum

- Heapsort
- binäre Suche
- Code-Komprimierung (z.B. Huffman-Baum)

Binärer Suchbaum

Definition binärer Suchbaum:

für jeden Knoten K mit Schlüssel k gilt:

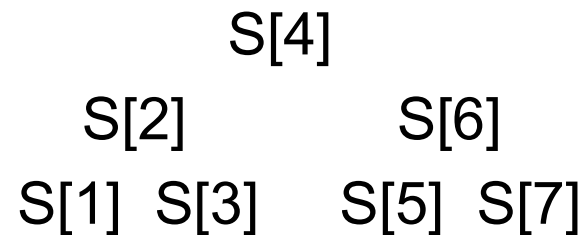
- alle Schlüssel links von K sind kleiner als k
- alle Schlüssel rechts von K sind größer oder gleich als k

äquivalente Definition:

- Ein binärer Suchbaum ist ein Binärbaum, dessen Inorder-Durchlauf die Schlüsselwerte in aufsteigender Reihenfolge wiedergibt.

Binärer Suchbaum

- Ausgehend von der binären Suche sind binäre Suchbäume nur eine anschauliche Darstellung
- Für die binäre Suche gilt: $\text{next} = (\text{high} + \text{low})/2$
- Veranschaulichung eines Arrays $S[1..7]$ durch einen binären Baum:



Binärer Suchbaum

- Die Blätter des Baumes stellen die Intervalle dar, die kein Element in dem Datenarray S enthalten.
- erfolglose Suchoperationen enden in den Blättern.
- Da die Blätter sich aus den Daten in S ergeben, müssen sie nicht explizit abgespeichert werden.

Binärer Suchbaum

- Komplexität bei der binären Suche: Unterscheide
 - Array- und Zeiger-Darstellung
- Search-Operation
 - Array- und Zeiger-Darstellung: $O(\log_2 n)$
- Für die Insert- und Delete-Operationen ist die Array-Implementierung problematisch (vgl. Tabelle)
 - Insert erfordert das Verschieben eines Teils des Arrays, und daher gilt dann für die Array-Implementierung $D(n) \in O(n)$
- Ausweg: Suchbaum durch Zeiger statt durch ein Array realisieren
 - Insert erfordert nur die Veränderung zweier Pointervariablen $O(1)$
 - Suche $S(n) \in O(\log_2 n)$

Binärer Suchbaum

```
template <class Tr>
void BinaryTree<Tr>::Search(TPosition<Tr> pos)
    TPosition<Tr> p;
    <Tr> r;
    pa=tree.Root();      // pa=Position der Wurzel
    while (pa.TPos())
    {
        r= GetNode(pos);
        s= GetNode(pa);
        if (r.key < s.key)
            pa = Left(pa);
        else
            pa = Right(pa);
    } //Search endet in einem Knoten pa, falls r.key = s.key
```

Balancierte Binärbäume

Ziele balancierte Binärbäume:

- Für eine effektive Suche in $O(\log n)$ müssen die Binärbäume „ausbalanciert“ sein
- Degenerieren der Bäume zur linearen Liste soll verhindert werden
- Aufwand für Einfüge- und Lösch-Operationen wird bei balancierten Suchbäumen höher
- Baum mit N Knoten soll Höhe $O(\log N)$ haben, damit Suchen in logarithmischer Zeit möglich ist
- AVL-Bäume sind die historisch ersten, die diesen Bedingungen gerecht werden.

AVL-Bäume

Entstehung AVL- Baum:

- 1962 vorgestellt von den Mathematikern
- Georgi M. **Adelson-Velski** und Jewgeni M. **Landis**
- älteste Datenstruktur für balancierte Binärbäume

AVL-Bäume

Definition AVL-Baum:

Ein binärer Suchbaum ist AVL-ausgeglichen oder höhenbalanciert:

- wenn für jeden Knoten p des Baumes gilt, dass sich die Höhe des linken Teilbaumes von der Höhe des rechten Teilbaumes von p höchstens um 1 unterscheidet.

Folgerung:

- AVL-Bäume können nicht zu linearen Listen degenerieren.
- Ein AVL-Baum der Höhe 1 hat 2 Knoten und ein AVL-Baum der Höhe 2 mit minimaler Knotenzahl hat 4 Knoten.
- Einen AVL-Baum der Höhe $h+2$ mit minimaler Knotenzahl erhält man, wenn man je einen AVL-Baum mit Höhe $h+1$ und h mit minimaler Knotenzahl zu einem Baum der Höhe $h+2$ zusammenfügt.

AVL-Bäume

Aus der Konstruktion eines AVL-Baums der Höhe h mit minimaler Knotenzahl folgt mit $\text{Fib}(n)$ = n -te Fibonacci-Zahl:

- Ein AVL-Baum mit Höhe h hat wenigstens $\text{Fib}(h+1)$ Blätter
- die Anzahl der Blätter eines AVL-Baumes wächst exponentiell mit der Höhe. (zur Erinnerung: $\text{Fib}(n) \in O(2^n)$ und $\text{Fib}(n) \in \Omega(2^{n/2})$)

Damit gilt für den AVL-Baum folgende Höhenbeschränkung:

- Ein AVL-Baum mit N Blättern hat eine Höhe $h \leq \log N$
- Ein AVL-Baum mit N Knoten hat eine Höhe $h \leq \log N$

Suchen, Einfügen und Entfernen von Schlüsseln im AVL-Baum

- Weil die Höhe logarithmisch beschränkt ist:
 - Suchen mit $O(\log N)$ möglich
- Einfügen und Löschen
 - Kommt der Schlüssel nicht im Suchbaum vor, läuft der Algorithmus in ein Blatt.
 - Einfügen oder Löschen von Schlüsselwerten kann zu unausgewogenem Baum führen

Suchen, Einfügen und Entfernen von Schlüsseln im AVL-Baum

Falls beim Einfügen oder Löschen AVL-Eigenschaft verletzt wird, muss diese anschließend geprüft und AVL-Ausgeglichenheit wieder hergestellt werden:

1. Prüfe AVL-Eigenschaft:
 - Laufe von der Einfügestelle zur Wurzel zurück und prüfe an jedem Knoten, ob die Höhendifferenz zwischen linkem und rechtem Teilbaum noch innerhalb der Grenzen liegt.
2. AVL-Eigenschaft wieder herstellen
 - Falls AVL-eigenschaft verletzt ist:
 - Führe geeignete **Rotationen** durch
 - Dadurch Ausgleichen der Höhendifferenz
 - AVL-Eigenschaft wird wieder hergestellt, Suchbaumeigenschaft bleibt erhalten.

Randomisierte Suchbäume

- zufälliges Einfügen von N Schlüsseln in einen leeren binären Suchbaum
- Wahrscheinlichkeit für degenerierten Suchbaum ist gering
- Erwartungswerte für die durchschnittliche Suchpfadlänge für einen zufällig erzeugten binären Suchbaum mit N Schlüsseln sind in der Größenordnung $O(\log N)$
- Falls die Schlüssel nicht zufallsverteilt sind Schlüsselwerte künstlich zufällig verteilen (Randomisieren)
- Vergabe eines zweiten zufällig generierten Schlüsselwerts
- Verarbeitung von Knoten mit 2 Komponenten nötig (z.B. Treap)

Treaps

Gegeben sei eine Menge von Knoten S . Jeder Knoten x aus S hat zwei Komponenten:

- Schlüsselkomponente $x.key$
- Prioritätskomponente $x.priority$

Definition Treap:

- Ein Treap ist ein binärer Suchbaum für die Schlüsselkomponenten.
- Ein Treap besitzt gleichzeitig Heapeigenschaft für die Prioritäten der Elemente aus S :
 - Söhne haben höhere Prio als Vater
- Treap muss keine vollständiger Binärbaum sein

Treaps

Treap = Hybridstruktur

1. Es gilt die Eigenschaft von binären Suchbäumen (Trees)

Suchbaumbedingung:

für jedes Element y im linken Teilbaum von x ist $y.key < x.key$
und für jedes Element z im rechten Teilbaum von x ist $x.key \leq z.key$.

2. Eigenschaften von Vorrangwarteschlangen (Heaps)

Heapbedingung:

Falls alle Söhne x eines Knoten w gilt:

$x.priority \geq w.priority$, d.h. alle Söhne haben höhere Priorität

B-Baum

Nach Prof R. Bayer, TUM

Definition:

Ein B-Baum der Ordnung m ist ein Baum mit folgenden Eigenschaften:

- Alle Blätter haben die gleiche Tiefe
- Jeder Knoten mit Ausnahme der Wurzel und der Blätter hat wenigstens $m/2$ Söhne
- Die Wurzel hat wenigstens 2 Söhne
- Jeder Knoten hat höchstens m Söhne
- Jeder Knoten mit i Söhnen hat $i-1$ Schlüssel
- Die Anzahl der Blätter ist um eins größer als die Anzahl der Schlüssel.
- Höhe des Baums: $h < \log(N)$

B-Bäume

Suchen eines Schlüssels im B-Baum:

- Start in der Wurzel
- Lineare Suche im Knoten
- Falls Schlüssel im Knoten nicht vorhanden -> gehe zum entsprechenden Sohn

B-Bäume

Einfügen eines Schlüssels im B-Baum:

- Suche Einfügeposition
- Falls Knoten weniger als m Schlüssel $\rightarrow$ Einfügen des Schlüssels
- Falls Knoten maximal besetzt $\rightarrow$ Splitting des Knotens

B-Bäume

Löschen eines Schlüssels im B-Baum:

- Lösche Schlüssel -> zu viele Söhne
- Neuverteilung der Schlüssel in den Söhnen